

Organización de Precios mediante Insertion Sort

Solución algorítmica incremental e iterativa para el ordenamiento eficiente de listas de precios de productos en tiendas.



Presentación Universitaria



Estructuras de Datos



Análisis de Costos $T(n)$

Planteamiento del Problema

Necesidad Comercial

Una tienda requiere organizar los precios de sus productos de forma dinámica e interactiva.

- La cantidad de productos **n** es variable y definida por el usuario.
- Se requiere ingresar los precios individuales de forma estructurada.
- Soporte para múltiples criterios de visualización según la estrategia de ventas.

Opciones de Ordenamiento

1. Orden Ascendente

Muestra los productos desde el menor costo hasta el mayor (foco en productos económicos).

2. Orden Descendente

Muestra los productos desde el mayor costo hasta el menor (foco en productos premium).

☰ Especificaciones del Sistema



Entrada

- **n:** Cantidad total de productos.
- **Precios:** n valores numéricos.
- **Opción:** 1 (Ascendente) o 2 (Descendente).



Proceso

- Validación de datos iniciales.
- Ejecución de **Insertion Sort**.
- Reordenamiento in-place del arreglo.



Salida

- Lista final de precios ordenada correctamente.
- Presentación clara según preferencia del usuario.

🛡️ **Restricciones del Sistema:** • n debe ser mayor que 0 | • Precios deben ser valores numéricos válidos | • Opción $\in \{1, 2\}$

Estrategia de Solución

1. Entrada

Solicitar n y registrar lista de precios

2. Config

Seleccionar tipo de ordenamiento

3. Sort

Aplicar Insertion Sort

4. Salida

Desplegar lista de precios ordenada

Naturaleza del Algoritmo

Insertion Sort construye progresivamente una sublista ordenada en la parte izquierda del arreglo. En cada iteración, toma un elemento no ordenado y lo inserta en su posición exacta.

Clasificación Técnica

- **Técnica:** Incremental (agrega un elemento por paso).
- **Implementación:** Iterativa (ciclos for y while anidados).
- **Algoritmo:** Insertion Sort (Ordenamiento por Inserción).

⚙️ ¿Cómo funciona Insertion Sort?

Ejemplo paso a paso (Orden Ascendente) con precios iniciales: **[5000, 2000, 8000, 3000]**

Inicio:

5000

2000

8000

3000

Paso 1
(clave=2000):

2000

5000

8000

3000

Paso 2
(clave=8000):

2000

5000

8000

3000

Paso 3
(clave=3000):

2000

3000

5000

8000

🔑 1. Toma la **clave**

🔍 2. Compara hacia la izquierda

↔️ 3. Desplaza elementos mayores

⬇️ 4. Inserta en hueco

↕ Dos Formas de Ordenar

↗ Orden Ascendente

Condición del bucle while:

```
precios[j] > clave
```

Resultado: Menor a Mayor

2000

3000

5000

8000

↘ Orden Descendente

Condición del bucle while:

```
precios[j] < clave
```

Resultado: Mayor a Menor

8000

5000

3000

2000

El núcleo del algoritmo permanece idéntico; únicamente se conmuta el operador de comparación (> por <).

</> Pseudocódigo — Insertion Sort

```
for i = 1 to n - 1
    clave = precios[i]
    j = i - 1

    while j ≥ 0 and precios[j] > clave
        precios[j + 1] = precios[j]
        j = j - 1

    precios[j + 1] = clave
```

Aspectos Clave

- El ciclo externo avanza desde el segundo elemento hasta el final ($i = 1 \dots n-1$).
- Se extrae la clave para comparar con la sublista izquierda.
- El ciclo interno **while** desplaza los elementos a la derecha mientras sean mayores a la clave.
- **Nota:** Para orden descendente, simplemente se reemplaza el operador $>$ por $<$ en la condición del **while**.

Implementación en Python (1/2)

```
# 1. Lectura de datos de entrada
n = int(input("Ingrese cantidad de productos: "))

precios = []

for i in range(n):
    precio = float(input(f"Precio del producto {i + 1}: "))
    precios.append(precio)

# 2. Selección del modo de ordenamiento
print("\nTipo de ordenamiento:\n1. Ascendente\n2. Descendente")
opcion = int(input("Seleccione una opción: "))
```

Captura de Datos

- Se solicita la dimensión n de la lista de precios.
- Bucle interactivo para capturar cada precio individual.
- Soporte para números reales mediante conversión `float`.
- Menú interactivo para capturar el modo de ordenamiento deseado.

Implementación en Python (2/2)

3. Algoritmo Insertion Sort con ambas opciones

```
for i in range(1, len(precios)):
    clave = precios[i]
    j = i - 1

    if opcion == 1:
        while j ≥ 0 and precios[j] > clave:
            precios[j + 1] = precios[j]
            j -= 1
    elif opcion == 2:
        while j ≥ 0 and precios[j] < clave:
            precios[j + 1] = precios[j]
            j -= 1

    precios[j + 1] = clave
```

Diagrama de Flujo





Análisis — Descomposición de Costos

Pseudocódigo	¿Qué hace?	Costo	Veces
for i = 1 to n - 1	Recorre los precios desde el segundo elemento hasta el último.	c_1	n
clave = precios[i]	Guarda el precio que se va a insertar en su posición correcta.	c_2	$n - 1$
j = i - 1	Ubica j en el elemento anterior al precio actual.	c_3	$n - 1$
while j ≥ 0 and precios[j] > clave	Compara los precios y determina si es necesario mover elementos.	c_4	$\sum t_i$
precios[j + 1] = precios[j]	Mueve el precio mayor una posición hacia la derecha.	c_5	$\sum(t_i - 1)$
j = j - 1	Retrocede una posición para continuar las comparaciones.	c_6	$\sum(t_i - 1)$
precios[j + 1] = clave	Coloca el precio en su posición correcta.	c_7	$n - 1$

Explicación y Función T(n)

$$T(n) = c_1 n + c_2 (n - 1) + c_3 (n - 1) + c_4 \sum t_i + c_5 \sum (t_i - 1) + c_6 \sum (t_i - 1) + c_7 (n - 1)$$

n

Cantidad total de productos/precios a ordenar.

c₁, c₂ ... c₇

Costo constante de ejecución de cada instrucción simple.

n - 1

Iteraciones requeridas al iniciar desde el 2° elemento.

Σ (Sumatoria)

Suma de iteraciones del ciclo interno en todas las pasadas.

t_i

Evaluaciones de la condición del while en la iteración i.

t_i - 1

Ejecuciones efectivas del cuerpo interno del while.

Casos del Algoritmo



Mejor Caso

Precios previamente ordenados.

[1000, 2000, 3000, 4000]

Mínimo de desplazamientos ($t_i = 1$).

Complejidad: $O(n)$



Caso Promedio

Precios en orden aleatorio.

[3000, 1000, 4000, 2000]

Desplazamientos moderados.

Complejidad: $O(n^2)$



Peor Caso

Precios en orden totalmente inverso.

[5000, 4000, 3000, 2000]

Máximo desplazamiento ($t_i = i$).

Complejidad: $O(n^2)$

Complejidad Temporal y Espacial

Complejidad Temporal (Big-O)

- **Mejor Caso:** $O(n)$ — lineal cuando la entrada ya está ordenada.
- **Caso Promedio:** $O(n^2)$ — comportamiento cuadrático general.
- **Peor Caso:** $O(n^2)$ — máximo número de comparaciones e intercambios.

Diferencia clave: **T(n)** da el costo exacto detallado por instrucción; **Big-O** representa el crecimiento asintótico superior.

Complejidad Espacial

$O(1)$

Espacio Auxiliar Adicional

Insertion Sort es un algoritmo In-Place:

- Ordena la lista modificando directamente la estructura original.
- Solo utiliza variables auxiliares simples (i, j, clave).
- La lista de entrada no se duplica en memoria.

▶ Ejemplo Completo de Ejecución

Entrada de prueba: $n = 5$ | Precios = [5000, 2000, 8000, 3000, 1000]

↑? Opción 1: Ascendente

Objetivo: Productos más económicos primero.

1000

2000

3000

5000

8000

Resultado final ordenado de menor a mayor.

↓? Opción 2: Descendente

Objetivo: Productos más costosos primero.

8000

5000

3000

2000

1000

Resultado final ordenado de mayor a menor.

🚩 Conclusiones

✓ Puntos Clave del Proyecto

- Se implementó exitosamente el algoritmo **Insertion Sort** para organizar listas dinámicas de precios.
- Soporta doble modalidad: ascendente y descendente ajustando únicamente la condición de comparación.
- Estrategia iterativa e incremental con complejidad temporal en el peor caso de **$O(n^2)$** y mejor caso de **$O(n)$** .
- Uso altamente eficiente del espacio auxiliar con **$O(1)$** memoria extra (In-Place).

“

"Insertion Sort es una solución sencilla, intuitiva y sumamente eficiente para conjuntos pequeños o parcialmente ordenados de datos."

Análisis de Algoritmos